



Assignment-2

Data Processing on Big Dataset with Databricks Spark

Name: Shreyash Narayane

Student ID: 25934391

2025 SPRING

94693 – Big Data Engineering

Master of Data Science and Innovation

University of Technology of Sydney



databricks

Table of Contents

Introduction	2
Part 1: Data Ingestion and Preparation	2
1. Download the notebook and run it on Databricks Free Edition	2
2. Download and manually import the location referential csv	2
3. Databricks Configuration	3
4. Exploring Dataset & Data Cleaning	4
5. Count the total number of rows for both green and yellow taxis	5
6. Combine Yellow and Green Taxi Dataset Together	5
7. Combine the new DataFrame with the location data	5
8. Save the final cleaned and joined dataset as a table	5
Part 2: Business Questions	6
1. Monthly Trip Analysis	6
2. Trip duration, distance and speed by taxi colour	6
3. Total trips, average distance, average amount & total amount	7
4. Revenue Share of Top 10 Borough Pairs (2024 only)	7
5. Percentage of Trips, Tips received	8
6. Percentage of Trips with Tips $\geq$ \$15	8
7. Compute Average speed (km/h) and distance per dollar (km/\$) by bins of durations	9
8. Target duration bin to maximise income	9
Part 3: Machine Learning (using SKLEARN)	10
Conclusion	13
References	14

Introduction

The purpose of this assignment is to process and analyze the New York City taxi trip data in Databricks using Big Data tools (SQL, Spark) and to predict the total amount of the trip with ML models. The workflow demonstrates three stages of work: (i.) ingestion and cleaning of large-scale taxi trip data, (ii.) SQL-based business questions answered with descriptive analytics, and (iii.) predictive modeling of trip fares using machine learning in scikit-learn.

The data represents more than 990 million rows spanning green and yellow taxi trips. Each trip record includes pickup and drop-off details, time stamps, passenger counts, fare-related fields, and location identifiers. By progressively refining the data, we obtain a dependable “bde.trips_final” table, which forms the basis for SQL aggregation and ML modeling.

Part 1: Data Ingestion and Preparation

1. Download the notebook and run it on Databricks Free Edition

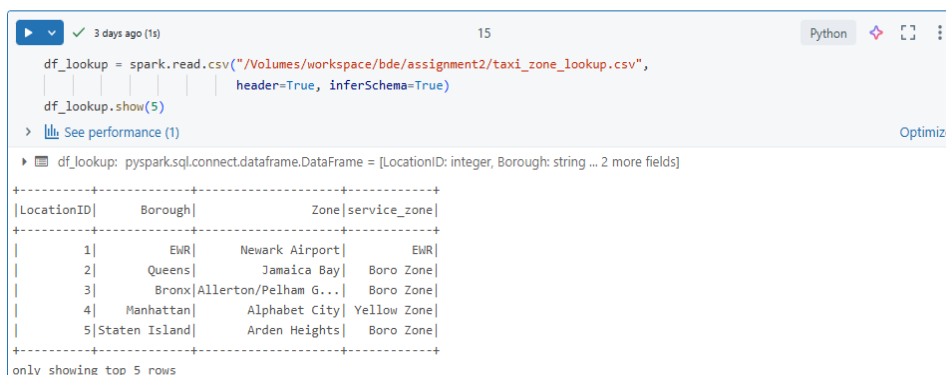
The notebook file was downloaded and uploaded in Databricks. It contained pre-written cells to download the “Yellow taxi” and “Green Taxi” data from Google Drive (using IDs), in parquet format saving them at path : “/Volumes/workspace/bde/assignment2/ ”

Two DataFrames were: green_df = 83484688 rows and yellow_df = 907982776 rows, whereas the total count = 991467464.

2. Download and manually import the location referential csv

The location referential csv: “taxi_zone_lookup.csv” was uploaded at the same path: “/Volumes/workspace/bde/assignment2/ ”

A DataFrame “df_lookup” was created from it for future use.



```
df_lookup = spark.read.csv("/Volumes/workspace/bde/assignment2/taxi_zone_lookup.csv",
                           header=True, inferSchema=True)
df_lookup.show(5)
```

> See performance (1)

df_lookup: pyspark.sql.connect.dataframe.DataFrame = [LocationID: integer, Borough: string ... 2 more fields]

LocationID	Borough	Zone	service_zone
1	Elmhurst	Newark Airport	Elmhurst
2	Queens	Jamaica Bay	Boro Zone
3	Bronx	Allerton/Pelham G...	Boro Zone
4	Manhattan	Alphabet City	Yellow Zone
5	Staten Island	Arden Heights	Boro Zone

only showing top 5 rows

3. Databricks Configuration

- i. Connect a “general compute serverless” and open the configuration settings.
- ii. Then, select 32GB of memory.
- iii. Add the dependencies and then apply the changes as below:

The screenshot shows the Databricks Environment configuration page. At the top, there's a header with 'Environment' and a 'Send feedback' link. Below this, the 'Memory' section has a dropdown menu set to 'High (32GB)'. The 'Serverless usage policy' section has a dropdown menu set to 'None'. The 'Base environment' section has a dropdown menu set to 'Custom' and a text input field containing '/path/to/base_environment.yml'. The 'Environment version' section has a dropdown menu set to '3'. The 'Dependencies' section has a table with three rows: 'scikit-learn', 'numpy', and 'threadpoolctl==3.2.0'. Below the table is a '+ Add dependency' link. At the bottom, there's a 'Ready to apply' button and a 'Saved' status indicator.

Environment [Send feedback](#) [X](#)

Memory ⓘ

High (32GB) ▼

Serverless usage policy ⓘ [🔗](#)

None ▼

Base environment ⓘ

Custom ▼

/path/to/base_environment.yml [📁](#)

Environment version ⓘ 3 ▼

Dependencies ⓘ

Added Available

scikit-learn	⋮
numpy	⋮
threadpoolctl==3.2.0	⋮

[+ Add dependency](#) ⓘ

[Ready to apply](#)

✓ Saved [pip logs](#) [Apply](#)

- iv. Updated config setting : High (32GB) memory
- v. Added Dependencies:
 - scikit-learn
 - numpy
 - threadpoolctl==3.2.0.

4. Exploring Dataset & Data Cleaning

The following steps were taken to clean the dataset:

a. Trips finishing before the starting time:

A function “drop_finish_before_start” is used which filters out and removes the rows from the dataset where the drop-off time is before the pickup time and returns a clean dataset, which are invalid records for trip data. This ensures all trips have a logical time order.

b. Trips where the pickup/drop-off datetime is outside of the range:

The trips with pickup or drop-off timestamps that fall outside of a valid date range are removed using the "drop_out_of_range" function. This will improve the quality of the data and avoid analysis errors caused by outliers or corrupted timestamps by guaranteeing that the dataset only includes records within the anticipated time window.

c. Trips with negative speed:

Trips where the calculated speed (distance/duration) is negative, physically impossible and indicates data errors are filtered out using the "remove_negative_speed" function.

d. Trips at very high speed:

Another function “remove_high_speed” is used to filter out trips with implausibly high speeds, which are due to data errors (e.g., incorrect timestamps or distances). The speed cap (SPEED_CAP_KMH) defines the maximum allowed speed (which was set to 100 km/h). Any trip with calculated speed above this cap is removed to ensure data quality.

e. Trips that are travelling too short or too long (duration wise):

Some trips have recorded noticeably short durations like less than a minute or more than 180 minutes, which are likely to be data errors or outliers. Hence, a function “remove_invalid_duration” is used to filter out trips with unrealistic durations. This helps ensure the dataset only contains trips with plausible durations.

f. Trips that are travelling too short or too long (distance wise):

A function “remove_invalid_distance_km” is used which filters out rows with trip distances (less than 0.1 km or more than 50 km).

g. Passenger counts:

In general cases, number of passengers in a car (depending upon the type) usually ranges between 1 to 9. Hence, a function “keep_passenger_1_to_6” is used to filter out rows where “passenger_count” is not between 1 and 6 (inclusive),

These filters are essential in large-scale taxi data. Erroneous entries, though small in percentage, can skew averages like speed or revenue per trip, leading to misleading insights.

5. Count the total number of rows for both green and yellow taxis

```
green_after4 = green_df_4g.count()
yellow_after4 = yellow_df_4g.count()

print(f"Green taxi latest count: {green_after4:,}")
print(f"Yellow taxi latest count: {yellow_after4:,}")
print(f"Combined total rows: {yellow_count + green_count:,}")
> See performance \(2\)
```

```
Green taxi latest count: 79,088,369
Yellow taxi latest count: 880,703,600
Combined total rows: 991,467,464
```

6. Combine Yellow and Green Taxi Dataset Together

Both the datasets were compared to identify common and exclusive columns. The columns were normalized and renamed for simplicity and maintaining consistency. The datasets were union into a single DataFrame.

7. Combine the new DataFrame with the location data

Both pickup and drop-off were joined to the lookup to obtain “pu_borough” and “do_borough”.

8. Save the final cleaned and joined dataset as a table

The resulting dataset was saved as “bde.trips_final”.

9. Count the total number of rows of your final table

Row count = 959791969

```
print(f"Row count of final table {TABLE_NAME} :", spark.table(TABLE_NAME).count())
> See performance \(1\)
```

```
Row count of final table bde.trips_final : 959791969
```

Part 2: Business Questions

1. Monthly Trip Analysis

The query summarizes taxi trips: calculating total trips, average passengers, busiest day & hour, average amount paid per trip & per passenger for each month.

The results show that trip volumes become significant after 2014. On average, rides carry about 1-2 passengers, and fares remain steady at \$19 per trip and at \$16 per passenger. Thursdays and Fridays dominate as the busiest days, while evenings around 6–7 PM are peak travel times, reflecting commuter and leisure demand.

Table	+							
	A _C ym	Σ ₃ total_trips	A _C busiest_dow	Σ ₃ busiest_hour	1.2 avg_passengers	1.2 avg_total_paid_per_trip	1.2 avg_paid_per_passenger	
1	2009-01	1222	Thursday	0	1.6	17.82	15.03	
2	2010-09	203	Thursday	1	1.05	17.41	16.88	
3	2011-01	2	Monday	23	1	12.8	12.8	
4	2011-02	1	Tuesday	0	1	15.8	15.8	
5	2012-09	3	Thursday	19	1	9.79	9.79	
6	2014-01	14438389	Friday	19	1.69	14.34	11.63	
7	2014-02	13925644	Saturday	19	1.68	14.48	11.78	
8	2014-03	16553179	Saturday	19	1.68	14.64	11.91	
9	2014-04	15770482	Wednesday	19	1.68	14.93	12.12	
10	2014-05	16026941	Friday	19	1.68	15.5	12.55	
11	2014-06	14988731	Sunday	19	1.68	15.5	12.56	
12	2014-07	14224602	Thursday	19	1.68	15.17	12.28	
13	2014-08	13706585	Friday	19	1.68	16.09	13.12	
14	2014-09	14563256	Tuesday	19	1.66	15.54	12.66	
15	2014-10	15624200	Friday	19	1.66	15.54	12.68	

2. Trip duration, distance and speed by taxi colour

The query calculates the average, median, minimum, and maximum trip duration (minutes), distance (kilometers), and speed (km/h) for both yellow and green taxis.

The results indicate that the trip data of yellow and green taxis are comparable, with average durations of 13–14 minutes and distances of 4.7–4.9 km. Median speeds differ slightly, with green taxis averaging a bit higher (18.5 km/h vs 16.6 km/h for yellow). Data quality filters are used by capping the maximum trip duration at 180 minutes and the maximum speed at 100 km/h. While trip lengths and durations are same for both, green taxis are a bit faster.

A _C color	.00 duration_avg_min	.00 duration_median_min	.00 duration_min_min	.00 duration_max_min	1.2 distance_avg_km	1.2 distance_median_km
green	13.52	10.50	1.00	180.00	4.75	3.09
yellow	14.36	11.25	1.00	180.00	4.87	2.74

1.2 distance_min_km	1.2 distance_max_km	1.2 speed_avg_kmh	1.2 speed_median_k...	1.2 speed_min_kmh	1.2 speed_max_kmh
0.11	49.99	20.4	18.54	0.04	99.99
0.11	49.99	18.88	16.58	0.04	100

3. Total trips, average distance, average amount & total amount

For each taxi color, pickup and drop-off borough, month, weekday, and hour, the query calculates the total number of taxi trips, average trip distance, average amount per trip, and total revenue.

The results highlight clear spatiotemporal travel patterns: trip volumes are highest in densely connected boroughs and peak during daytime and evening hours. Average trip distances stay short (3–5 km), while amount remain steady around \$10–12 per ride. Total amount closely follows trip volumes, meaning boroughs and times with more rides also dominate earnings.

	A_C^B color	A_C^B pu_borough	A_C^B do_borough	s_3^B month_num	A_C^B day_of_week	s_3^B hour_of_day	s_3^B total_trips	1.2 avg_distance_km	1.2 avg_total_amount	1.2 total_amnt
1	green	Bronx	Bronx	1	Friday	0	1126	3.64	10.44	11759.92
2	green	Bronx	Bronx	1	Friday	1	987	3.64	10.7	10557
3	green	Bronx	Bronx	1	Friday	2	709	3.9	11.23	7959.46
4	green	Bronx	Bronx	1	Friday	3	527	4.19	11.21	5907.38
5	green	Bronx	Bronx	1	Friday	4	453	4.33	11.58	5244.91
6	green	Bronx	Bronx	1	Friday	5	457	4.41	10.88	4970.66
7	green	Bronx	Bronx	1	Friday	6	750	4.54	11.26	8446.33
8	green	Bronx	Bronx	1	Friday	7	2242	3.97	11.27	25261.45
9	green	Bronx	Bronx	1	Friday	8	3526	3.73	11.28	39775.91
10	green	Bronx	Bronx	1	Friday	9	2752	3.95	11.18	30760.48
11	green	Bronx	Bronx	1	Friday	10	2349	4.11	11.63	27312.65
12	green	Bronx	Bronx	1	Friday	11	2307	4.25	11.88	27414.03
13	green	Bronx	Bronx	1	Friday	12	2382	4.19	11.79	28075.16
14	green	Bronx	Bronx	1	Friday	13	2389	4.03	11.53	27552.11
15	green	Bronx	Bronx	1	Friday	14	2922	3.96	11.75	34340.44

4. Revenue Share of Top 10 Borough Pairs (2024 only)

The query returns the top ten pairs along with their revenue and percentage share of the year's total taxi revenue after filtering taxi trips to 2024, aggregating “total_amount” by pickup-dropoff borough pair, and ranking pairs by “total_amount”.

According to the results, more than 61% of 2024 revenue will come from trips between Manhattan-Manhattan. Queens-Manhattan (16.5%) and Manhattan-Queens (6.7%) are the next two major flows. The top three pairs generate almost 85% of revenue, show how a limited number of routes account for the majority of NYC taxi revenue.

	A_C^B pu_borough	A_C^B do_borough	1.2 revenue_usd	1.2 share_prct	s_3^B rk
1	Manhattan	Manhattan	629667332.58	61.74	1
2	Queens	Manhattan	168581831.68	16.53	2
3	Manhattan	Queens	67925573.53	6.66	3
4	Queens	Brooklyn	36747514.89	3.6	4
5	Manhattan	Brooklyn	31243393.79	3.06	5
6	Queens	Queens	29688575.29	2.91	6
7	Manhattan	EWB	11681154.3	1.15	7
8	Queens	Unknown	9261353.73	0.91	8
9	Manhattan	Unknown	5895442.04	0.58	9
10	Queens	Bronx	5384850.96	0.53	10

5. Percentage of Trips, Tips received

The query calculates the percentage of trips where passengers tipped the driver, by dividing the count of trips tipped by total trips and multiplying by one hundred.

Tipping is a common but not universal practice, as evidenced by the fact that 63.05% of trips included a tip. This suggests a cultural norm of tipping in NYC taxis, though a sizeable proportion of passengers (around 37%) still choose not to tip.

```
SELECT
  ROUND((100.0 * SUM(CASE WHEN tip_amount > 0 THEN 1 ELSE 0 END)) / COUNT(*), 2)
  AS prcnt_trips_with_tips
FROM bde.trips_final;
```

> [See performance \(1\)](#)

Table	
.00 prcnt_trips_with_tips	
1	63.05

6. Percentage of Trips with Tips ≥\$15

The query looks only at trips where a tip was given and calculates the percentage of those tips that were at least \$15. (Measuring high-value tips percentage).

The results show that only 0.80% of tipping trips involve tips of \$15 or more, meaning large tips are quite rare. While most riders do tips, the majority give modest amounts, and high-value gratuities form only a small fraction of overall driver earnings.

```
SELECT
  ROUND(
    100.0 * SUM(CASE WHEN tip_amount >= 15 THEN 1 ELSE 0 END)
    / SUM(CASE WHEN tip_amount > 0 THEN 1 ELSE 0 END), 2
  ) AS prcnt_tips_grtr_thn_15
FROM bde.trips_final;
```

> [See performance \(1\)](#)

Table	
.00 prcnt_tips_grtr_thn_15	
1	0.80

7. Compute Average speed (km/h) and distance per dollar (km/\$) by bins of durations

The query computes the average speed (km/h) and distance per dollar (km/\$) for each duration bins.

The results state that as duration increases, average speed rises (highways/freeways), but km-per-dollar also grows, reducing earning efficiency (lower km-per-dollar means higher earnings per distance). Thus, longer trips cover more ground but bring lower earnings per km compared to short city rides.

	^A ₀ duration_bin	1.2 avg_speed_kmh	1.2 avg_km_per_dollar
1	Under 5 mins	19.75	0.1629
2	5–10 mins	17.2	0.2105
3	10–20 mins	17.9	0.2574
4	20–30 mins	21.35	0.305
5	30–60 mins	25.73	0.3699
6	At least 60 mins	22.08	0.5035

8. Target duration bin to maximise income

The query computes: average fare, tip rate/amount, revenue per hour, plus speed, km/\$, and volume share for each bin. The trip length that maximizes a driver's earnings is then displayed by ranking the bins according to revenue per hour.

The results indicate that trips lasting under 5-minutes generate the highest hourly revenue (\$123.2) although they account for only 14% of overall rides. The 5–10-minute bin has much higher trip volume (30%) marginally lower hourly revenue (\$82.6) makes it more viable driver strategy. Despite higher fares, hourly earnings are steadily reduced by longer trips.

Since the 5–10-minute duration bin provides the best balance of high hourly revenue and a sizable trip share (with consistent demand), this duration bin must be "primarily targeted" by the taxi drivers.

	^A ₀ duration_bin	1.2 avg total per trip usd	.00 avg duration min	1.2 revenue per hour usd	.00 tip rate pct	1.2 avg tip usd	1.2 avg speed kmh	1.2 avg km per dollar	^A ₃ trips in bin	.00 trip share pct
1	Under 5 mins	7.25	3.53	123.2	54.91	0.77	19.75	0.1629	134172829	13.98
2	5–10 mins	10.21	7.42	82.59	61.85	1.12	17.2	0.2105	285003099	29.69
3	10–20 mins	47.28	39.27	72.23	66.07	5.28	25.73	0.3699	70843613	7.38
4	20–30 mins	16.21	14.18	68.6	65.49	1.79	17.9	0.2574	339611747	35.38
5	30–60 mins	27.48	24.11	68.41	66.53	3.11	21.35	0.305	122170103	12.73
6	At least 60 mins	68.42	73.23	56.06	58.68	7.05	22.08	0.5035	7990578	0.83

Part 3: Machine Learning (using SKLEARN)

Objective

To predict “total_amount” of a trip and calculate RMSE, using features available at trip start.

DataFrame

- i. In Part 2 for Question 3. C, the result CSV was saved and manually uploaded at the path: "/Volumes/workspace/bde/assignment2/"
- ii. Used Pandas to read the CSV file located at the path and then stored the contents into a DataFrame called “q3c_df”.
- iii. Loaded the full “bde.trips_final” table with Spark, took a 1% sample, and converted it into a Pandas DataFrame as “trip_df”. Extracted the month number, day of week, and hour of day as new features in the “trip_df”.
- iv. Split the dataset “trip_df” into training (before Oct 2024) and testing (Oct–Dec 2024) sets, creating separate DataFrames “train_df” and “test_df” respectively.

Features

- i. Categorical features selected = 'color', 'pu_borough', 'do_borough', 'day_of_week'
- ii. Numerical features selected = 'passenger_count', 'trip_distance', 'month_num', 'hour_of_day', 'tip_amount', 'payment_type'
- iii. Final features list = 'color', 'pu_borough', 'do_borough', 'day_of_week', 'passenger_count', 'trip_distance', 'month_num', 'hour_of_day', 'tip_amount', 'payment_type'
- iv. Target Feature = 'total_amount'

Model preparation

Separated the “train_df” and “test_df” datasets into features (X) and target (y), based on the selected column lists.

Preprocessing

A ColumnTransformer used here that applies one-hot encoding to categorical columns, passes numerical columns through unchanged, and drops any others, preparing features for modeling.

Baseline Model

Using average fares per borough-pair-hour (from Part 2 Q3.c) as prediction gave a Baseline RMSE = 26.60. This approach represents the simplest possible estimator: predicting each trip's fare using historical averages for its geographic and temporal slot. It is quick to calculate and simple to explain, but it has a high prediction error because it ignores individual trip factors like distance or passenger count. Any significant model should perform better than the baseline, which is only useful as a benchmark.

Ridge Regression Model

Ridge regression cut the error in half compared to the baseline, yielding a Test RMSE = 12.99. In high-dimensional feature spaces (like many one-hot encoded boroughs and time slots), Ridge uses L2 regularization, which reduces coefficients and aids in managing multicollinearity. It scales well, trains quickly, and can still be understood using coefficients. Its linear form is the trade-off; complex non-linear effects, like surges at times, are only partially captured, which restricts the final accuracy.

Random Forest Regressor

The Random Forest model outperformed Ridge by a significant margin, with a Test RMSE = 11.01, and nearly reduced the baseline error by two-thirds. It flexibly captures complex nonlinear relationships and interaction effects between features (which others could not), such as how distance interacts with borough-pair to determine fares, by averaging across multiple decision trees. Because of its adaptability, it can accommodate a range of trip fare patterns, from quick local rides to more involved interborough travel. This increase in predictive accuracy shows that Random Forest is the best model for deployment because it can produce dependable results and generalize well.

However, training required 12 minutes and consumed more memory. This complexity makes it a bit difficult to use in low-latency or resource-limited environments.

Overall Model Performance

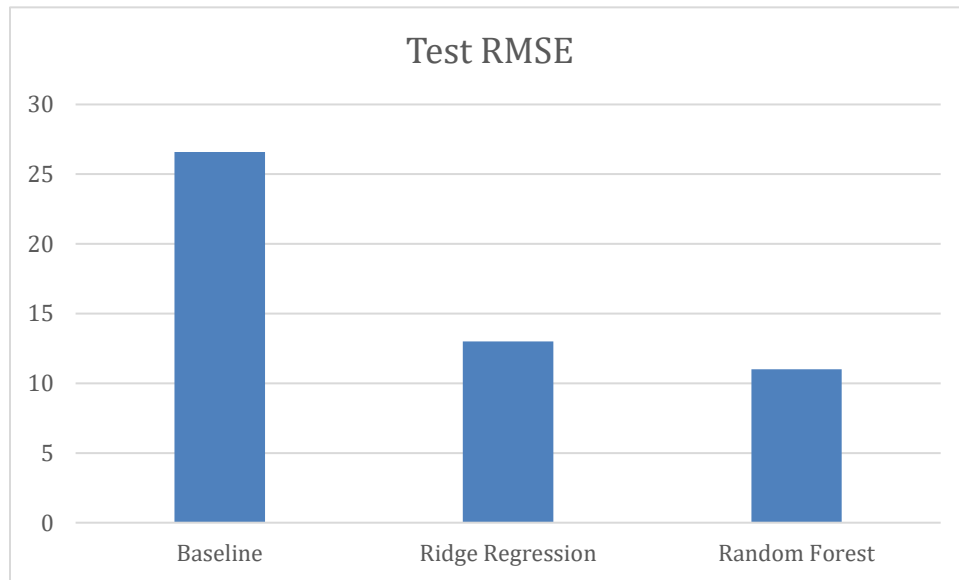
Random Forest provides the most accurate predictions and represents a clear step forward in practical deployment. Ridge is still an alternative, but Random Forest demonstrates the strongest capability for capturing fare dynamics on a scale.

Model	Test RMSE
Baseline	26.60035
Ridge Regression	12.992214
Random Forest	11.013514

Best Model

Random Forest Regressor is the best model as it achieves the lowest Test RMSE (11.01), a clear margin over Ridge (12.99) and dramatically better than the baseline (26.60).

Random Forest is a bit complex and took 12 minutes end-to-end versus Ridge's less than 1 minute. The accuracy gain (1.98 RMSE vs Ridge) justifies the extra compute when prediction accuracy is the priority and inference volume is manageable.



Recommendation

- For deployment aimed at maximizing predictive accuracy: **Random Forest**.
- For resource-constrained use cases: **Ridge** remains a backup, though not necessary if compute is sufficient.
- Both models dramatically outperform the baseline, proving that trip-level features contain strong predictive signal.

Conclusion

This assignment used data from NYC taxis to show how big data engineering, analytics, and machine learning can be combined. Through robust data preparation and cleaning, reliable insights highlighting demand cycles, geographic concentration, and passenger behavior were obtained for almost one billion trips. Business questions revealed Consistent temporal and spatial patterns, including the rarity of very high tips, the dominance of weekend evenings, and the concentration of revenue within the top inter-borough routes. According to the analysis of trip duration, trips lasting five to ten minutes optimize driver income efficiency and provide the best balance of high revenue per hour and large trip share (with steady demand). The Random Forest Regressor outperformed Ridge Regression in predictive machine learning modeling, achieving the lowest RMSE. While Ridge was faster and simpler, Random Forest, despite being a bit complex and time-consuming, proved to be more effective overall by capturing nonlinear patterns and providing superior accuracy for fare prediction.

Overall, the project demonstrates the importance of scalable data pipelines and predictive models in enhancing business strategy and operational decision-making for taxi services.



References

- NYC Taxi & Limousine Commission trip records (via provided brief).
- taxi_zone_lookup.csv (via provided brief).
- scikit-learn (Ridge, RandomForestRegressor), NumPy, Pandas.
- Databricks Spark for data preparation and SQL.

